

GITHUB ENGINEERING · 2026-08-17

GitHub 8 · 17 宕机复盘

一次容量峰值，如何沿着服务网格、负载均衡与客户端重试，演变成持续
7 小时 47 分钟的全站故障。

来源: <https://github.blog/news-insights/company-news/the-august-17-outage-and-the-work-ahead/>

先看影响：错误率最高达 50%

7h 47m

13:28–21:15 UTC

≈20%

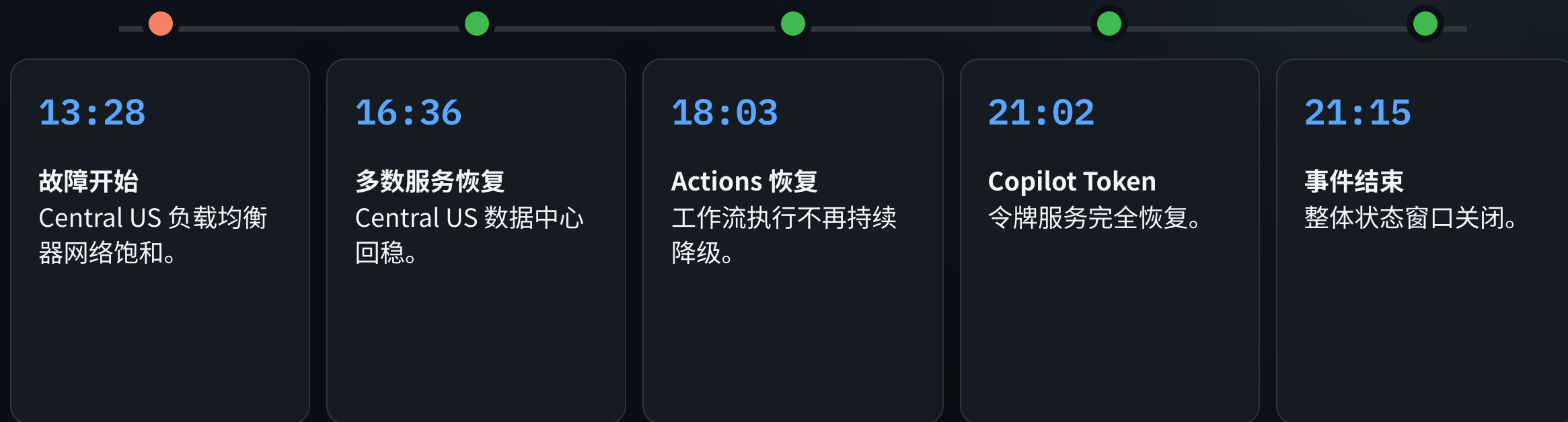
Web 与 API 峰值错误率

≈50%

归档与 Raw 内容下载错误率

这不是单点页面故障：认证链路受阻后，Issues、Pull Requests、Actions、Copilot 与企业身份服务一起承压。

恢复并非同一时刻完成



认证路径把影响面连在了一起



协作与 API

Issues、Pull Requests、Web/API 请求出现错误与高延迟。



内容下载

Archive、codeload 与 raw-content 的失败率更高。



企业身份

SAML/OIDC、SCIM 与 Team Sync 同步受到影响。



GitHub Actions

Actions 降级持续更久，公共步骤定义成为依赖点。



数据驻留

GHEC Data Residency workflow 依赖 GitHub.com 上的公共定义。



GitHub Copilot

Token Service 被客户端重试风暴拖入延迟恢复。

区域转移有效，但没有切断放大回路



Central US

网络饱和、HAProxy flow 耗尽；部分失败流量被迁出。



Northern Virginia

迁入流量最初成功承接，证明跨区服务路径可用。

但 Copilot 的失败响应触发客户端重试，压力随后在 Northern VA 重新聚集。

根因是一条容量级联链

01

新流量峰值

Central US 的入口负载超过历史容量包络。



02

Istio sidecar 满载

Pod 达到并发上限，成为最先显现的瓶颈。



03

扩容策略看错指标

策略监控 host service，却没有监控 sidecar 限制。



04

4 台 HAProxy 耗尽 flow

网关认证路径延迟飙升，故障扩散到更多产品。

关键点：高可用节点仍然在线，并不等于它们还有可用的流量处理余量。

重试把局部过载变成正反馈

$$\text{总请求} = \text{原始请求} \times (1 + \text{重试})$$

当失败概率随负载上升时，“再试一次”会制造更多失败，而不是提高成功率。

乐观网关重试继续向内部负载均衡器施压。

延迟响应让客户端误判请求未完成，重复发送。

缺少抖动与预算会让大量调用方同步重试，形成脉冲流量。

Copilot Token Service: 流量被放大约 10×

正常



7-9K RPS

故障峰值



70-100K
RPS

单个失败的 token 操作可能产生许多额外请求，并进入循环。

≈ 10×

VS Code 潜在重试 bug 带来的请求放大

恢复策略：先止血，再逐站点放量

01

暂停异常节点

同时暂停 4 台 HAProxy，立即带来广泛恢复。



02

减少网关重试

通过 PR 临时降低 gateway retry logic。



03

返回明确的 403

在负载均衡器阻断 token 请求，避免触发更危险的响应路径。



04

渐进恢复流量

按站点逐步 ramp up，让调用方先恢复成功。

为什么恢复花了更久

外部噪声：抓取攻击

针对 codeload 端点的多次 scraping attacks 与真实用户流量叠加，使容量判断和恢复更困难。

- 下载请求单位成本较高
- 峰值期间更容易争抢连接与带宽
- 恶意与故障放大流量需要分别识别

内部盲点：监控对象不完整

系统看见 host service 的指标，却没有把 sidecar 与 flow 容量视为同一条端到端路径。

- 组件健康不代表路径健康
- 区域切换也必须设置接收上限
- 恢复操作需要防止客户端继续放大

后续工作：把容量、重试与故障域一起治理



修正扩容策略

把 sidecar 并发与容量纳入 autoscaling。



审计 Istio 限制

检查 request、concurrency 与 scaling 配置。



约束重试行为

复核网关与客户端的预算、退避和抖动。



修复 VS Code

消除放大 Copilot token 流量的潜在循环。



强化区域防护

改进 LB 容量监控与区域 failover safeguards。

真正的韧性，不是“还能重试”，而是过载时能主动减载、隔离并受控恢复。